# Universidade da Coruña

Grado en Ciencia e Ingeniería de Datos


Análisis e Interpretación de Datos Audiovisuales — Práctica 1

---

# Clasificación de Imágenes Portuarias mediante Redes Neuronales Convolucionales

---


**Autor:**
Enrique Pardo García


**Fecha:**
Abril de 2026

# Índice


**1. Introducción** **2**

**2. Dataset** **2**

**3. Arquitecturas de los Modelos** **2**
3.1. SmallCNN (entrenamiento desde cero) . . . . . 2
3.2. ResNet-18 con pesos ImageNet (transfer learning) . . . . . 3

**4. Configuraciones Experimentales** **3**
4.1. Parámetros de entrenamiento base . . . . . 3
4.2. Espacio de configuraciones — Tarea 1 . . . . . 3
4.3. Espacio de configuraciones — Tarea 2 . . . . . 4
4.4. Pipeline de Data Augmentation . . . . . 4

**5. Métricas de Evaluación y Robustez** **5**

**6. Resultados — Tarea 1: Ship Detection** **5**
6.1. Tabla global de resultados . . . . . 5
6.2. Comparación entre grupos . . . . . 6
6.3. Curvas de entrenamiento — configuración ganadora . . . . . 6
6.4. Configuración más robusta . . . . . 7
6.5. Configuración más inestable . . . . . 7

**7. Resultados — Tarea 2: Docked Detection (Fine-tuning)** **8**
7.1. Tabla global de resultados . . . . . 8
7.2. Comparación entre grupos (Tarea 2) . . . . . 8
7.3. Curvas de la mejor configuración (Tarea 2) . . . . . 8

**8. Análisis de Robustez** **9**
8.1. Efecto del tipo de modelo . . . . . 9
8.2. Efecto de la data augmentation . . . . . 9
8.3. Efecto del val_split . . . . . 10
8.4. Efecto del aug_level . . . . . 10
8.5. Configuración más robusta: análisis específico . . . . . 10

**9. Conclusiones** **10**

# 1. Introducción

El presente trabajo aborda un problema de **clasificación binaria de imágenes** en el contexto de un puerto marítimo. El objetivo principal es determinar, a partir de imágenes aéreas o de vigilancia, si en una imagen aparece un barco o no (*Tarea 1: Ship Detection*), y en una segunda etapa, si el barco detectado se encuentra atracado o no (*Tarea 2: Docked Detection* mediante *fine-tuning*).

La motivación es doble: por un lado, se exploran dos estrategias de entrenamiento bien diferenciadas — entrenamiento desde cero (*from scratch*) frente al uso de pesos preentrenados en ImageNet (*transfer learning*) —; por otro, se estudia de forma sistemática el impacto que tienen la **proporción de datos de validación** (*val_split*) y el **nivel de aumento de datos** (*aug_level*) sobre la precisión y la robustez del modelo.

Para garantizar que los resultados no son producto del azar, cada configuración se repite **3 veces con semillas distintas** y se reportan las métricas como $\mu \pm \sigma$. Además se implementa un indicador de robustez propio definido como la media de las desviaciones típicas de las tres métricas principales (*accuracy*, F1, AUC-ROC) a lo largo de los distintos *runs*.

Debido al tamaño del modelo entrenado, el archivo comprimido con la práctica completa ha sido subido a OneDrive y puede descargarse a través del siguiente enlace: Enlace a OneDrive.

# 2. Dataset

El conjunto de datos proporcionado consiste en imágenes RGB de resolución variable almacenadas en la carpeta `P1-Material/images/`, acompañadas de dos ficheros CSV:

- `ship.csv`: etiqueta binaria que indica si la imagen contiene un barco (1) o no (0). Se utiliza para la Tarea 1.
- `docked.csv`: subconjunto de imágenes con barcos, etiquetado según si el barco está atracado (1) o no (0). Se utiliza para la Tarea 2.

Las imágenes se redimensionan a $224 \times 224$ píxeles, normalización estándar de ImageNet ($\mu = [0{,}485, 0{,}456, 0{,}406]$, $\sigma = [0{,}229, 0{,}224, 0{,}225]$), adecuada tanto para el modelo preentrenado como para el entrenamiento desde cero.

La división en entrenamiento y validación se realiza de forma aleatoria controlada por semilla, variando la proporción de validación entre tres valores: 10 %, 20 % y 30 %.

# 3. Arquitecturas de los Modelos

Se emplean dos arquitecturas para la Tarea 1 y una de ellas para la Tarea 2:

## 3.1. SmallCNN (entrenamiento desde cero)

Red convolucional ligera de diseño propio con cuatro bloques Conv2d $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool con 32, 64, 128 y 256 filtros respectivamente. El clasificador aplica

*AdaptiveAvgPool2d*$(4 \times 4)$, una capa densa de 512 unidades con Dropout(0.4) y la salida binaria.

### 3.2. ResNet-18 con pesos ImageNet (transfer learning)

Se utiliza ResNet-18 preentrenada en ImageNet-1k (`ResNet18_Weights.IMAGENET1K_V1`). La única modificación es sustituir la capa `fc` final por:

$$\text{Dropout}(0{,}3) \rightarrow \text{Linear}(512, 1) \tag{1}$$

En la Tarea 2 (*fine-tuning*) se parte de los pesos de la mejor configuración de la Tarea 1 y se reentrena el modelo completo sobre el subconjunto de barcos con la etiqueta de atracado.

## 4. Configuraciones Experimentales

### 4.1. Parámetros de entrenamiento base

Cuadro 1: Hiperparámetros de entrenamiento para las dos tareas.

| **Parámetro** | **Tarea 1 (Scratch)** | **Tarea 1/2 (Pretrained)** |
|---|---|---|
| Épocas | 40 | 30 |
| Learning rate | $1 \times 10^{-3}$ | $3 \times 10^{-4}$ |
| Weight decay | $1 \times 10^{-4}$ | $1 \times 10^{-4}$ |
| Batch size | 32 | 32 |
| Optimizador | AdamW | AdamW |
| Scheduler | CosineAnnealingLR | CosineAnnealingLR |
| Función de pérdida | BCEWithLogitsLoss | BCEWithLogitsLoss |
| Criterio best model | Val AUC | Val AUC |
| Nº runs por config | 3 | 3 |

### 4.2. Espacio de configuraciones — Tarea 1

Para la Tarea 1 se prueban **18 configuraciones** resultado del producto cartesiano:

$$\{\text{scratch}, \text{pretrained}\} \times \{\text{sin aug}, \text{aug_level} = 1{,}0, \text{aug_level} = 1{,}5\} \times \{\text{val_split} = 0{,}1, 0{,}2, 0{,}3\}$$

Cuadro 2: Todas las configuraciones de la Tarea 1 (Ship Detection).

| **Nombre** | **Modelo** | **Aug** | **aug_level** | **val_split** |
|---|---|---|---|---|
| scratch_noaug_val10 | SmallCNN | No | — | 0.1 |
| scratch_noaug_val20 | SmallCNN | No | — | 0.2 |
| scratch_noaug_val30 | SmallCNN | No | — | 0.3 |
| scratch_aug10_val10 | SmallCNN | Sí | 1.0 | 0.1 |
| scratch_aug10_val20 | SmallCNN | Sí | 1.0 | 0.2 |
| scratch_aug10_val30 | SmallCNN | Sí | 1.0 | 0.3 |
| scratch_aug15_val10 | SmallCNN | Sí | 1.5 | 0.1 |
| scratch_aug15_val20 | SmallCNN | Sí | 1.5 | 0.2 |
| scratch_aug15_val30 | SmallCNN | Sí | 1.5 | 0.3 |
| pretrained_noaug_val10 | ResNet-18 | No | — | 0.1 |
| pretrained_noaug_val20 | ResNet-18 | No | — | 0.2 |
| pretrained_noaug_val30 | ResNet-18 | No | — | 0.3 |
| pretrained_aug10_val10 | ResNet-18 | Sí | 1.0 | 0.1 |
| pretrained_aug10_val20 | ResNet-18 | Sí | 1.0 | 0.2 |
| pretrained_aug10_val30 | ResNet-18 | Sí | 1.0 | 0.3 |
| pretrained_aug15_val10 | ResNet-18 | Sí | 1.5 | 0.1 |
| pretrained_aug15_val20 | ResNet-18 | Sí | 1.5 | 0.2 |
| pretrained_aug15_val30 | ResNet-18 | Sí | 1.5 | 0.3 |

## 4.3. Espacio de configuraciones — Tarea 2

La Tarea 2 solo emplea ResNet-18 preentrenada (parte del mejor modelo obtenido en la Tarea 1) y reduce el espacio a **9 configuraciones**:

$$\{\text{sin aug}, \text{aug_level} = 1{,}0, \text{aug_level} = 1{,}5\} \times \{\text{val_split} = 0{,}1, 0{,}2, 0{,}3\}$$

## 4.4. Pipeline de Data Augmentation

El parámetro `aug_level` escala proporcionalmente todas las transformaciones:

Cuadro 3: Transformaciones de aumentación y su escala con `aug_level`.

| **Transformación** | **aug_level = 1.0** | **aug_level = 1.5** |
|---|---|---|
| Resize + RandomCrop | $256 \times 256 \to 224 \times 224$ | igual |
| RandomHorizontalFlip (p) | $p = 0{,}50$ | $p = 0{,}75$ |
| RandomVerticalFlip (p) | $p = 0{,}20$ | $p = 0{,}30$ |
| ColorJitter (brightness) | $\pm 0{,}40$ | $\pm 0{,}60$ |
| ColorJitter (contrast) | $\pm 0{,}40$ | $\pm 0{,}60$ |
| ColorJitter (saturation) | $\pm 0{,}30$ | $\pm 0{,}45$ |
| ColorJitter (hue) | $\pm 0{,}05$ | $\pm 0{,}075$ |
| RandomGrayscale (p) | $p = 0{,}05$ | $p = 0{,}075$ |
| RandomRotation (grados) | $\pm 15$ř | $\pm 22$ř |

# 5. Métricas de Evaluación y Robustez

Se evalúan tres métricas sobre el conjunto de validación del mejor modelo de cada *run* (seleccionado por máximo AUC de validación):

- **Accuracy**: fracción de ejemplos clasificados correctamente.
- **F1-Score**: media armónica de precisión y recall; robusta ante posibles desbalances entre clases.
- **AUC-ROC**: área bajo la curva ROC; mide la capacidad discriminativa del modelo independientemente del umbral de clasificación.

El **indicador de robustez** se define como:

$$\text{Robustez} = \frac{1}{3}\left(\sigma_{\text{Acc}} + \sigma_{\text{F1}} + \sigma_{\text{AUC}}\right) \tag{2}$$

donde $\sigma$ es la desviación típica de la métrica entre los 3 *runs*. Un valor bajo indica que el modelo es estable frente a la inicialización aleatoria.

Adicionalmente, durante el entrenamiento se monitorizó la **desviación típica de la pérdida de validación en ventanas de 5 épocas** (`val_loss_std`), que detecta cambios bruscos indicativos de inestabilidad o sobreajuste incipiente.

# 6. Resultados — Tarea 1: Ship Detection

## 6.1. Tabla global de resultados

Cuadro 4: Resultados globales (media ± desv. típica sobre 3 *runs*) en la Tarea 1. Robustez según Ec. 2 (menor = más estable).

| **Configuración** | **Accuracy** | **F1** | **AUC** | **Robustez** |
|---|---|---|---|---|
| scratch_noaug_val10 | 0,8506 ± 0,0709 | 0,8811 ± 0,0633 | 0,9466 ± 0,0353 | 0.0565 |
| scratch_noaug_val20 | 0,8851 ± 0,0163 | 0,9094 ± 0,0194 | 0,9708 ± 0,0022 | 0.0126 |
| scratch_noaug_val30 | 0,9167 ± 0,0142 | 0,9376 ± 0,0132 | 0,9600 ± 0,0127 | 0.0133 |
| scratch_aug10_val10 | 0,8276 ± 0,0745 | 0,8686 ± 0,0661 | 0,9412 ± 0,0360 | 0.0589 |
| scratch_aug10_val20 | 0,8563 ± 0,0430 | 0,8928 ± 0,0355 | 0,9383 ± 0,0173 | 0.0319 |
| scratch_aug10_val30 | 0,8561 ± 0,0298 | 0,8995 ± 0,0225 | 0,9342 ± 0,0088 | 0.0204 |
| scratch_aug15_val10 | 0,8046 ± 0,0586 | 0,8257 ± 0,0958 | 0,9416 ± 0,0372 | 0.0639 |
| scratch_aug15_val20 | 0,8103 ± 0,0614 | 0,8674 ± 0,0453 | 0,9133 ± 0,0034 | 0.0367 |
| scratch_aug15_val30 | 0,8258 ± 0,0298 | 0,8781 ± 0,0193 | 0,9100 ± 0,0148 | 0.0213 |
| pretrained_noaug_val10 | 0,9770 ± 0,0163 | 0,9797 ± 0,0144 | 1,0000 ± 0,0000 | 0.0102 |
| pretrained_noaug_val20 | 0,9770 ± 0,0081 | 0,9815 ± 0,0077 | 0,9980 ± 0,0021 | 0.0060 |
| green!15 pretrained_noaug_val30 | **0,9773 ± 0,0093** | **0,9824 ± 0,0079** | 0,9972 ± 0,0015 | 0.0062 |
| pretrained_aug10_val10 | 0,9195 ± 0,0325 | 0,9400 ± 0,0205 | 1,0000 ± 0,0000 | 0.0177 |
| pretrained_aug10_val20 | 0,9770 ± 0,0081 | 0,9814 ± 0,0077 | 0,9996 ± 0,0006 | 0.0055 |
| pretrained_aug10_val30 | 0,9583 ± 0,0142 | 0,9691 ± 0,0095 | 0,9978 ± 0,0011 | 0.0083 |
| blue!10 pretrained_aug15_val10 | 0,9655 ± 0,0000 | 0,9716 ± 0,0032 | 1,0000 ± 0,0000 | **0.0011** |
| pretrained_aug15_val20 | 0,9598 ± 0,0081 | 0,9679 ± 0,0081 | 0,9970 ± 0,0006 | 0.0056 |
| pretrained_aug15_val30 | 0,9205 ± 0,0643 | 0,9322 ± 0,0592 | 0,9979 ± 0,0018 | 0.0418 |

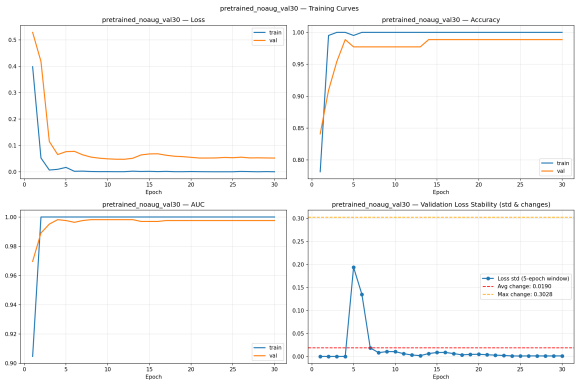
Mejor accuracy/F1 global  Configuración más robusta

## 6.2. Comparación entre grupos

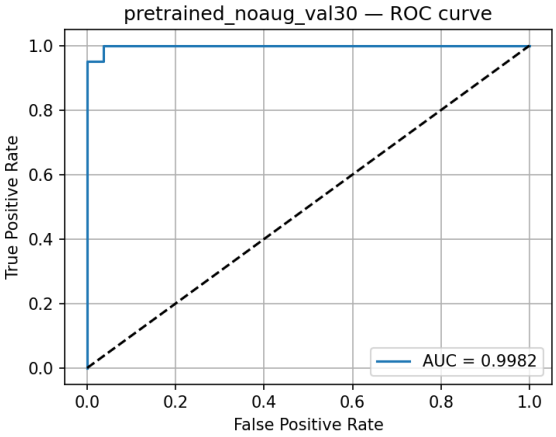
Cuadro 5: Métricas medias por grupo en la Tarea 1.

| Grupo | Acc media | F1 media | AUC media | Robustez |
|---|---|---|---|---|
| Scratch sin augmentation | 0.8841 | 0.9094 | 0.9591 | 0.0275 |
| Scratch con augmentation | 0.8301 | 0.8720 | 0.9298 | 0.0388 |
| Pretrained sin augmentation | 0.9771 | 0.9812 | 0.9984 | 0.0075 |
| Pretrained con augmentation | 0.9501 | 0.9604 | 0.9987 | 0.0133 |

## 6.3. Curvas de entrenamiento — configuración ganadora




(a) Historial de entrenamiento (mejor config).




(b) Curva ROC (mejor config).

Figura 1: `pretrained_noaug_val30`: curvas de entrenamiento y ROC (Acc = 0.9773, AUC = 0.9972).

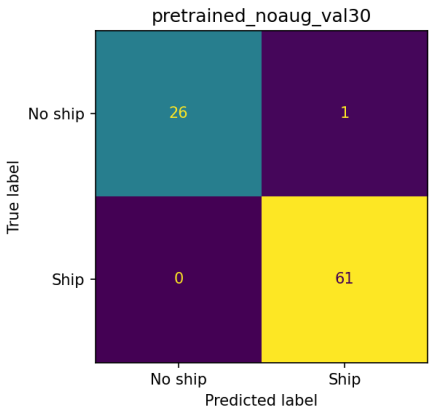
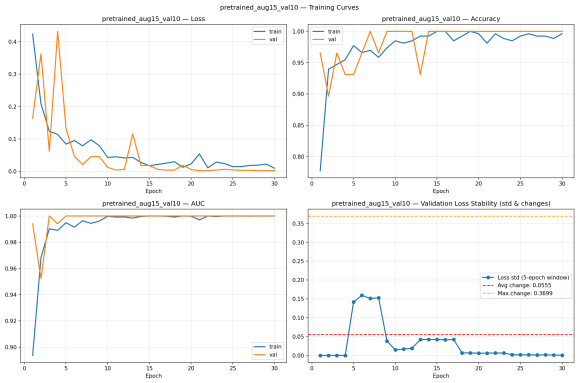
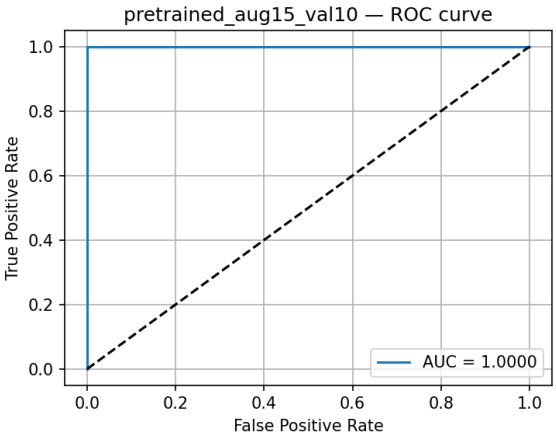



Figura 2: Matriz de confusión de `pretrained_noaug_val30` (run 1).

## 6.4. Configuración más robusta



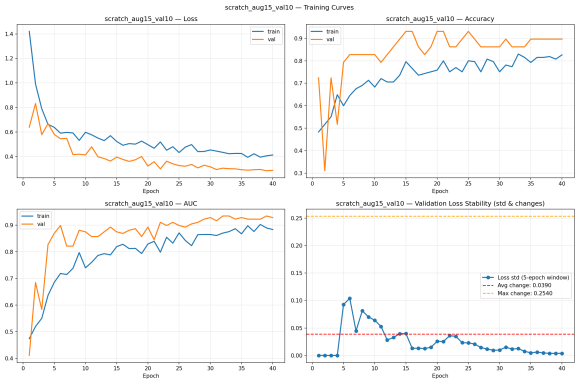

(a) Historial de entrenamiento (más robusta).



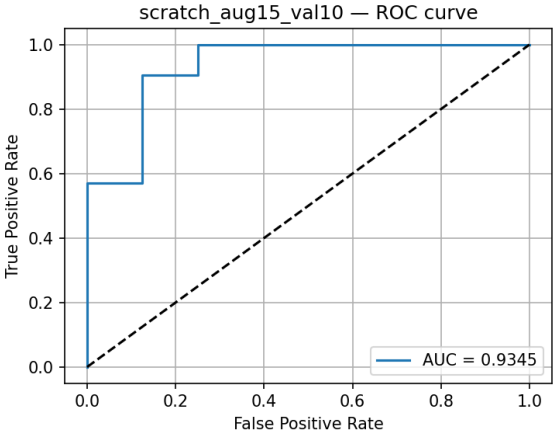

(b) Curva ROC (más robusta).

Figura 3: `pretrained_aug15_val10`: configuración más estable entre runs (Robustez = 0.0011, Acc = 0.9655 ± 0.0000).

## 6.5. Configuración más inestable




(a) Historial de entrenamiento (más inestable).




(b) Curva ROC (más inestable).

Figura 4: `scratch_aug15_val10`: peor configuración en robustez (Robustez = 0.0639, Acc = 0.8046 ± 0.0586).

# 7. Resultados — Tarea 2: Docked Detection (Fine-tuning)

## 7.1. Tabla global de resultados

Cuadro 6: Resultados del fine-tuning para detección de barcos atracados (Tarea 2).

| Configuración | Accuracy | F1 | AUC | Robustez |
|---|---|---|---|---|
| docked_noaug_val10 | 0,7407 ± 0,0524 | 0,5833 ± 0,1434 | 0,8662 ± 0,0172 | 0.0710 |
| green!15 docked_noaug_val20 | **0,8056 ± 0,0227** | 0,7366 ± 0,0454 | 0,8265 ± 0,0667 | 0.0449 |
| docked_noaug_val30 | 0,7939 ± 0,0309 | 0,7536 ± 0,0508 | 0,8325 ± 0,0578 | 0.0465 |
| docked_aug10_val10 | 0,7778 ± 0,0786 | 0,6707 ± 0,1040 | 0,9126 ± 0,0520 | 0.0782 |
| blue!10 docked_aug10_val20 | 0,7870 ± 0,0131 | 0,7071 ± 0,0186 | 0,8862 ± 0,0207 | **0.0175** |
| docked_aug10_val30 | 0,8000 ± 0,0148 | **0,7642 ± 0,0455** | 0,8468 ± 0,0338 | 0.0314 |
| docked_aug15_val10 | 0,7407 ± 0,0693 | 0,5945 ± 0,0521 | 0,7994 ± 0,1043 | 0.0752 |
| docked_aug15_val20 | 0,7222 ± 0,0454 | 0,6514 ± 0,0373 | 0,8275 ± 0,0138 | 0.0322 |
| docked_aug15_val30 | 0,7697 ± 0,0454 | 0,7318 ± 0,0424 | 0,8350 ± 0,0302 | 0.0393 |

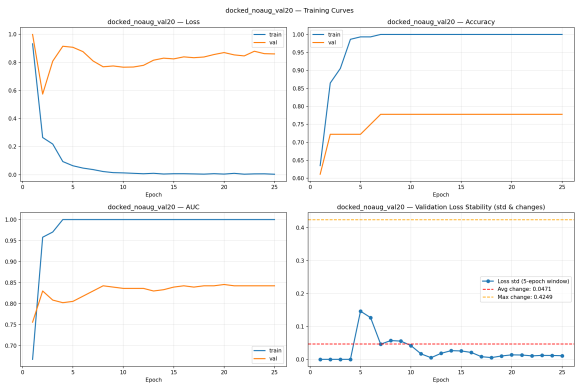
Mejor accuracy  Más robusta  F1 en negrita: mejor F1 global

## 7.2. Comparación entre grupos (Tarea 2)

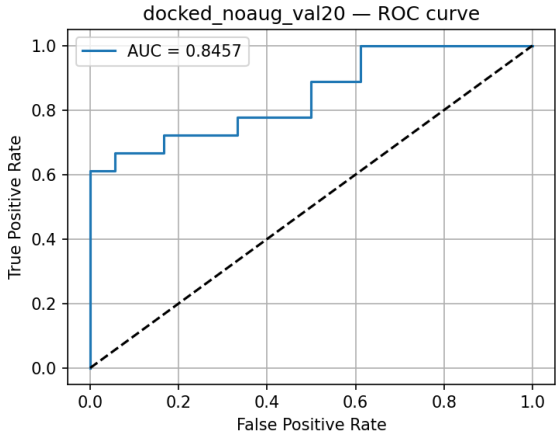
Cuadro 7: Comparación entre grupos en la Tarea 2.

| Grupo | Acc media | F1 media | AUC media | Robustez |
|---|---|---|---|---|
| Pretrained sin augmentation | 0.7801 | 0.6912 | 0.8417 | 0.0541 |
| Pretrained con augmentation | 0.7662 | 0.6866 | 0.8512 | 0.0456 |

## 7.3. Curvas de la mejor configuración (Tarea 2)




(a) Historial de entrenamiento.




(b) Curva ROC.

Figura 5: `docked_noaug_val20`: mejor accuracy en la Tarea 2 (Acc = 0.8056, AUC = 0.8265).

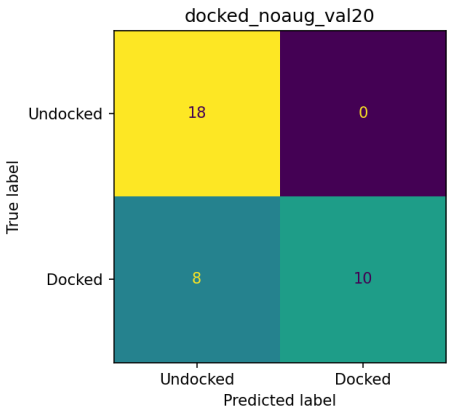



Figura 6: Matriz de confusión de `docked_noaug_val20` (run 1).

# 8. Análisis de Robustez

## 8.1. Efecto del tipo de modelo

El resultado más llamativo del experimento es la **clara superioridad del modelo preentrenado** sobre el entrenado desde cero, tanto en rendimiento como en estabilidad:

- Accuracy media: *scratch* = 0.857 vs *pretrained* = 0.964 (+12.4 puntos porcentuales).
- Robustez media: *scratch* = 0.0332 vs *pretrained* = 0.0104 (3× más estable).

Esto es esperable: los pesos ImageNet ofrecen representaciones de bajo nivel (bordes, texturas) ya optimizadas, lo que reduce la varianza de entrenamiento y acelera la convergencia hacia mejores mínimos en el espacio de pérdidas.

## 8.2. Efecto de la data augmentation

Contrariamente a lo que podría intuirse, **la augmentation no mejora la accuracy media** en ninguna de las dos tareas:

| | Con aug | Sin aug |
|---|---|---|
| Acc media (Tarea 1) | 0.890 | 0.931 |
| Robustez media (Tarea 1) | 0.0261 | 0.0175 |
| Acc media (Tarea 2) | 0.766 | 0.780 |
| Robustez media (Tarea 2) | **0.0456** | 0.0541 |

Una posible explicación es que el conjunto de datos es suficientemente homogéneo (imágenes portuarias tomadas en condiciones similares), por lo que introducir variabilidad artificial actúa como ruido en lugar de regularización. El modelo preentrenado ya dispone de representaciones robustas que no necesitan reforzarse con augmentation agresiva. Sin embargo, en la Tarea 2 la augmentation sí reduce la variabilidad entre runs (0.0456 <0.0541), lo que sugiere un efecto regularizador en el fine-tuning con datos más escasos.

## 8.3. Efecto del val_split

- En la Tarea 1, el mejor *val_split* global por accuracy es **0.2** (acc_media = 0.911 sobre 6 configs), y también el mejor en robustez (std_medio = 0.0164).
- En la Tarea 2, val_split = 0.3 da mejor accuracy media (0.788), pero val_split = 0.2 sigue siendo el más robusto (std_medio = 0.0315). Existe un **trade-off accuracy/robustez** que debe tenerse en cuenta.

Reducir demasiado el conjunto de validación (val_split = 0.1) genera estimaciones ruidosas que se traducen en alta varianza entre runs, mientras que reservar demasiado dato para validación (val_split = 0.3) puede perjudicar ligeramente el entrenamiento cuando el dataset no es muy grande.

## 8.4. Efecto del aug_level

Para las configuraciones con augmentation, aug_level = 1.0 supera sistemáticamente a aug_level = 1.5 tanto en accuracy como en robustez. La augmentation excesivamente agresiva (1.5) introduce una variabilidad que el modelo no consigue compensar en el número de épocas disponible, resultando en modelos menos precisos y más inestables.

## 8.5. Configuración más robusta: análisis específico

La configuración `pretrained_aug15_val10` obtuvo la robustez perfecta de **0.0011** (prácticamente sin variación entre los 3 runs), con accuracy de 0.9655 ± 0.0000 y AUC = 1.0000 ± 0.0000. Esta estabilidad extrema se debe probablemente a que el modelo converge siempre al mismo mínimo al combinar la fuerte regularización del aug_level alto con los pesos preentrenados. No obstante, su accuracy es 1.18 puntos menor que la configuración ganadora.

# 9. Conclusiones

Este trabajo ha analizado de forma sistemática el efecto de cuatro factores sobre la clasificación de imágenes portuarias: tipo de modelo (scratch vs. preentrenado), uso de augmentation, nivel de augmentation y proporción de validación. Las principales conclusiones son:

1. **El transfer learning es esencial**: ResNet-18 preentrenada supera a la CNN desde cero en más de 10 puntos porcentuales y es 3 veces más robusta. En este tipo de datasets pequeños/medianos, prescindir de los pesos preentrenados supone una penalización muy notable.
2. **La augmentation no siempre ayuda**: en este dataset la augmentation reduce ligeramente la accuracy en ambas tareas. Sí tiene un efecto regularizador sobre la variabilidad entre runs en la Tarea 2, por lo que puede considerarse si la reproducibilidad es más importante que el rendimiento absoluto.
3. **val_split = 0.2 como punto óptimo**: es el valor que mejor equilibra el aprovechamiento de datos para entrenamiento y la fiabilidad de la estimación del rendimiento en validación.

4. **La configuración ganadora** (`pretrained_noaug_val30`) alcanza Acc = 0.977, F1 = 0.982 y AUC = 0.997 en la Tarea 1, siendo un resultado muy sólido. La configuración más robusta (`pretrained_aug15_val10`) alcanza Acc $= 0.966 \pm 0.000$ con variabilidad prácticamente nula.